

GUIA RÁPIDO - MÉTODOS NUMÉRICOS PARA ENCONTRAR RAÍZES

1. COMPILAÇÃO E EXECUÇÃO

Na pasta raiz do projeto:

```
make          -> compila a aplicação definida em main.cpp
./main_app    -> executa a aplicação principal

make moodle   -> compila os exercícios em
test_moodle/test_moodle.cpp
./moodle_test -> executa os exercícios Moodle selecionados no
main

make test     -> compila os testes de números duais
./run_test_dual -> executa os testes de números duais
```

2. DEFININDO UMA FUNÇÃO

As funções devem receber e retornar Dual. Isso permite que o método de Newton calcule a derivada automaticamente.

Exemplo: $f(x) = x^2 - 2$

```
auto f = [](const Dual& x) {
    return x * x - 2;
};
```

As operações disponíveis incluem +, -, *, /, exp(x), log(x), sqrt(x), sin(x), cos(x), tan(x) e pow(base, x).

3. EXECUTANDO OS MÉTODOS

As funções dos métodos estão declaradas em solvers.hpp e retornam um SolverResult:

```
SolverResult bissecao(funcao, a, b, tolerancia);
SolverResult falsaPosicao(funcao, a, b, tolerancia, max_iteracoes);
SolverResult newton_raphson(funcao, chute_inicial, tolerancia,
max_iteracoes);
SolverResult secante(funcao, x0, x1, tolerancia, max_iteracoes);
```

Exemplos:

```
SolverResult r1 = bissecao(f, 1.0, 2.0, 1e-6);
SolverResult r2 = falsaPosicao(f, 1.0, 2.0, 1e-6, 100);
SolverResult r3 = newton_raphson(f, 1.5, 1e-6, 100);
SolverResult r4 = secante(f, 1.0, 2.0, 1e-6, 100);
```

Bisseção e falsa posição exigem um intervalo [a, b] que contenha mudança de sinal. Newton exige um chute inicial. Secante exige duas aproximações

iniciais.

4. IMPRIMINDO APENAS A SOLUÇÃO

Após executar um método, a raiz aproximada está em `result.root`:

```
SolverResult resultado = newton_raphson(f, 1.5, 1e-6, 100);
std::cout << "Raiz aproximada: " << resultado.root << '\n';
std::cout << "Iterações: " << resultado.iterations_used << '\n';
std::cout << "Status: " << resultado.get_status_message() << '\n';
```

5. IMPRIMINDO O RELATÓRIO DE UM RESULTADO

Inclua `reporter.hpp` e chame `report_method`. Ele apresenta status, iterações, critério de convergência, raiz, erros e histórico iterativo:

```
SolverResult resultado = newton_raphson(f, 1.5, 1e-6, 100);
report_method(resultado);
```

Também é possível informar o nome exibido no relatório e imprimir somente uma em cada N iterações do histórico:

```
report_method("Newton para  $x^2 - 2$ ", resultado, 2);
```

O terceiro argumento (`every`) vale 1 por padrão e controla a frequência de linhas exibidas: 1 imprime todas; 2 imprime uma a cada duas iterações.

6. EXECUTANDO E IMPRIMINDO O RELATÓRIO DIRETAMENTE

As funções abaixo executam o método e já exibem o relatório:

```
report_bissecao(f, 1.0, 2.0, 1e-6);
report_falsa_posicao(f, 1.0, 2.0, 1e-6, 100);
report_newton(f, 1.5, 1e-6, 100);
report_secante(f, 1.0, 2.0, 1e-6, 100);
```

Elas também retornam `SolverResult`, caso seja necessário acessar a raiz:

```
SolverResult resultado = report_secante(f, 1.0, 2.0, 1e-6, 100);
std::cout << resultado.root << '\n';
```

7. RELATÓRIO COMPARATIVO

Para comparar todos os métodos de uma vez, use:

```
report_all(f, 1.0, 2.0, 1e-6, 100);
```

O relatório mostra bisseção, falsa posição, Newton-Raphson e secante com quantidade de iterações, status, tipo de convergência e ordem estimada.

8. EXERCÍCIOS MOODLE

Os exercícios implementados estão em `test_moodle/test_moodle.cpp`. Para escolher quais rodar, altere o `main` desse arquivo. Exemplo:

```
int main() {  
    exercicio_11(1e-4);  
    exercicio_12(1e-4);  
    return 0;  
}
```

Depois, execute:

```
make moodle  
./moodle_test
```